



Research Paper



A penalty-type method for solving inverse optimal value problem in second-order conic programming

Yue Lu^{a,*}, Zheng-Peng Dong^a, Zhi-Qiang Hu^a, Hong-Min Ma^a, Dong-Yang Xue^b

^a School of Mathematical Sciences, Tianjin Normal University, Tianjin, 300387, China

^b School of Mechanical Engineering, Tianjin University of Commerce, Tianjin, 300134, China

ARTICLE INFO

Keywords:

Inverse optimal value problem
Second-order conic programming
Penalty-type method

ABSTRACT

This paper aims to consider a type of inverse optimal value problem in second-order conic programming, in which the parameter in its objective function needs to be adjusted under a given class that makes the corresponding optimal objective value closest to a target value. This inverse problem can be reformulated as a minimization problem with some second-order cone complementarity constraints. To tackle these bilinear constraints, we apply a penalty-type method and show that the associated penalty term is exact, which avoids the hurdles of penalty-type methods in the update strategy of the penalty parameter. Numerical results show that our method is suitable to solve the given inverse optimal value problem under different metric distances between the optimal value of the forward problem and a target objective counterpart.

1. Introduction

In the traditional optimization process, the data of parameters in the model is given, and the main task is to generate an optimal decision for a specified objective function with constraints. More specifically, consider an optimization problem of the form

$$\begin{aligned} \min_x \quad & f(x, \theta) \\ \text{s.t.} \quad & x \in \mathcal{X}(\theta), \end{aligned} \quad (1)$$

where $x \in \mathbb{R}^n$ is the decision variable, $\theta \in \mathbb{R}^m$ is the parameter from a given class $\Theta \subseteq \mathbb{R}^m$, $f(x, \theta) : \mathbb{R}^{n+m} \rightarrow \mathbb{R}$ and $\mathcal{X}(\theta) \subseteq \mathbb{R}^n$ are the objective function and the feasible set, respectively. We call problem (1) the forward problem. However, in some applications, one needs to consider the reverse situation, i.e., finding values of parameters in the forward problem that renders a given feasible decision optimal or makes the corresponding optimal objective value closest to a target value. In both cases, these models are known as inverse problems.

1.1. Literature reviews

The systematic study of inverse problems began in some shortest paths problems initially analyzed by Burton and Toint [4], whose results triggered a series of contributions on inverse network problems [5,27,26,2,3,10]. After that, several special

* Corresponding author.

E-mail addresses: jnjin403@sina.com (Y. Lu), dzp215839@163.com (Z.-P. Dong), hzhqiang42@163.com (Z.-Q. Hu), mahongminya@sina.com (H.-M. Ma), dongyangxue@tjcu.edu.cn (D.-Y. Xue).

<https://doi.org/10.1016/j.apnum.2024.02.004>

Received 25 May 2023; Received in revised form 12 January 2024; Accepted 5 February 2024

Available online 9 February 2024

0168-9274/© 2024 IMACS. Published by Elsevier B.V. All rights reserved.

inverse continuous optimization models appear in the literature [25,24,28,11,19–21,14,15]. These problems can be formulated as

$$\begin{aligned} \min_{\theta} \quad & d(\theta, \hat{\theta}) \\ \text{s.t.} \quad & \theta \in \Theta^{\text{inv}}(\hat{x}), \\ & \theta \in \Theta, \end{aligned}$$

where $\hat{\theta} \in \mathbb{R}^m$ is a nominal parameter value, $d(\theta, \hat{\theta})$ denotes the distance between θ and $\hat{\theta}$ under an appropriate metric and $\Theta^{\text{inv}}(\hat{x})$ is the inverse-feasible set that is the collection of θ for which a feasible decision $\hat{x} \in \mathcal{X}(\theta)$ is optimal to problem (1), i.e., $\Theta^{\text{inv}}(\hat{x}) := \{\theta \in \mathbb{R}^m : f(\hat{x}, \theta) \leq f(x, \theta), \forall x \in \mathcal{X}(\theta)\}$. One can refer to inverse linear programming problem [25,24], inverse quadratic programming problem [28], inverse second-order cone programming problem [11], inverse positive semidefinite cone programming problem [19–21,14,15] and references therein for more details.

Rather than presenting a feasible decision $\hat{x} \in \mathbb{R}^n$, a target value $\hat{v} \in \mathbb{R}$ is observed, and the associated inverse problem now turns to find a parameter $\theta \in \mathbb{R}^m$ such that the optimal value of the forward problem is closest to $\hat{v}$, which leads to the following model

$$\begin{aligned} \min_{\theta} \quad & d(v(\theta), \hat{v}) \\ \text{s.t.} \quad & \theta \in \Theta, \end{aligned} \quad (2)$$

where $v(\theta)$ denotes the optimal value of problem (1) and $d(v(\theta), \hat{v})$ denotes the distance between $v(\theta)$ and $\hat{v}$. The above problem is called the inverse optimal value problem.

There are some research works on inverse optimal value problems for combinatorial models such as the minimum spanning tree problem [12,18,22,23] and the shortest paths problem [29]. For the field of continuous optimization problems, Ahmed and Guan [1] discussed the linear programming problem, in which they proved that its inverse optimal value problem is NP-hard and got the optimal parameter by solving a series of linear and bilinear programming problems under some special assumptions. However, to our best knowledge, there is no discussion about inverse optimal value problems for other types of continuous forward counterparts.

1.2. Motivation

In the past three decades, a great deal of mathematical effort in conic programming has been devoted to the following second-order conic programming (SOCP) problem

$$\begin{aligned} \min_x \quad & \theta^T x \\ \text{s.t.} \quad & A_j x - b_j \in \mathcal{K}_{m_j}, \quad j = 1, 2, \dots, J, \end{aligned} \quad (3)$$

where $\theta, x \in \mathbb{R}^n$, $A_j \in \mathbb{R}^{m_j \times n}$, $b \in \mathbb{R}^{m_j}$ and $\mathcal{K}_{m_j}$ is a m_j -dimensional second-order cone, i.e., $\mathcal{K}_{m_j} := \left\{ y \in \mathbb{R}^{m_j} : \left(\sum_{i=2}^{m_j} y_i^2 \right)^{1/2} \leq y_1 \right\}$. For the sake of technical convenience, we will assume that these matrices $A_j \in \mathbb{R}^{m_j \times n}$ ($j = 1, 2, \dots, J$) are of full column rank. This model not only covers linear programming problems, convex quadratic programming problems, quadratically constrained convex quadratic programming problems, but also realistic problems such as filter design and antenna array weight design [13].

We present an example to demonstrate the importance of studying the inverse optimal value problem in SOCP.

Example 1.1 (The facility location problem [7]). Let $v_1, \dots, v_m \in \mathbb{R}^2$ be the locations of m villages and $p_1, \dots, p_m \in \mathbb{R}$ be the unit prices of roads. To find the location of a service station $s \in \mathbb{R}^2$ that makes the total cost of roads linking the station and the villages as small as possible, one needs to solve the following problem

$$\min_s \sum_{j=1}^m p_j \|s - v_j\|_2, \quad (4)$$

where $s := (s_1, s_2)^T \in \mathbb{R}^2$ and $v_j := (v_{j1}, v_{j2})^T \in \mathbb{R}^2$, $j = 1, 2, \dots, m$.

By introducing some auxiliary variables $t_1, t_2, \dots, t_m \in \mathbb{R}$, one can formulate problem (4) as

$$\begin{aligned} \min_{t_1, \dots, t_m, s_1, s_2} \quad & \sum_{j=1}^m p_j t_j \\ \text{s.t.} \quad & (t_j, s_1 - v_{j1}, s_2 - v_{j2})^T \in \mathcal{K}_3, \quad j = 1, 2, \dots, m. \end{aligned}$$

Notice that the above problem can be rewritten as problem (3) with

$$\begin{aligned} \theta &:= (p_1, \dots, p_m, 0, 0)^T \in \mathbb{R}^{m+2}, \quad x := (t_1, \dots, t_m, s_1, s_2)^T \in \mathbb{R}^{m+2}, \\ A_j &:= \begin{pmatrix} e_j^T \\ I_2 \end{pmatrix} \in \mathbb{R}^{3 \times (m+2)}, \quad b_j := (0, v_{j1}, v_{j2})^T \in \mathbb{R}^3, \quad j = 1, 2, \dots, m, \end{aligned}$$

where e_j is the j -th unit vector in $\mathbb{R}^m$.

Let $\hat{v}$ be the estimated total cost of problem (4). With the development of technology, the unit prices $p_1, \dots, p_m$ change dynamically, which are respectively defined in P_j , i.e., $p_j \in P_j$, $j = 1, 2, \dots, m$. Now, consider the update of road networks, the management department faces the issue of whether to adjust the location of the service station and redetermine the unit price of roads for each village. In this case, one needs to solve problem (2) with $\Theta := P_1 \times \dots \times P_m \times \{0\} \times \{0\}$, which is a typical inverse optimal value problem in second-order conic programming.

1.3. The problem description

In this paper, we consider the inverse optimal value problem in second-order conic programming, in which the term $v(\theta)$ in (2) is set to be the optimal value of SOCP defined as in (3). Notice that problem (2) has a bilevel structure due to the appearance of $v(\theta)$.

Assume that problem (3) has an attainable optimal value and the associated Slater's condition holds, i.e., there exists a vector $\tilde{x} \in \mathbb{R}^n$ such that $A_j \tilde{x} - b_j \in \text{int } \mathcal{K}_{m_j}$, $j = 1, 2, \dots, J$, where $\text{int } \mathcal{K}_m$ denotes the interior of $\mathcal{K}_m$. In the light of the conic duality theorem and the Karush-Kuhn-Tucker (KKT) condition of problem (3), our problem reduces to the following single-level optimization problem with linear second-order cone complementarity constraints

$$\begin{aligned} \min_{\theta, x, y_1, y_2, \dots, y_J} \quad & d(\theta^T x, \hat{v}) \\ \text{s.t.} \quad & \theta - \sum_{j=1}^J A_j^T y_j = 0, \theta \in \Theta, \\ & y_j^T (A_j x - b_j) = 0, j = 1, 2, \dots, J, \\ & A_j x - b_j \in \mathcal{K}_{m_j}, y_j \in \mathcal{K}_{m_j}, j = 1, 2, \dots, J. \end{aligned} \quad (5)$$

In order to tackle bilinear constraints $y_j^T (A_j x - b_j) = 0$, the associated penalized subproblem is considered, i.e.,

$$\begin{aligned} \min_{\theta, x, y_1, y_2, \dots, y_J} \quad & d(\theta^T x, \hat{v}) + \sigma \sum_{j=1}^J y_j^T (A_j x - b_j) \\ \text{s.t.} \quad & x \in \mathcal{F}, (\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}, \end{aligned} \quad (6)$$

where $\sigma \in \mathbb{R}$ is a positive penalty parameter and $\mathcal{F}, \mathcal{T}$ are respectively defined by

$$\begin{aligned} \mathcal{F} &:= \left\{ x : A_j x - b_j \in \mathcal{K}_{m_j}, j = 1, 2, \dots, J \right\}, \\ \mathcal{T} &:= \left\{ (\theta, y_1, y_2, \dots, y_J) : \theta \in \Theta, \theta - \sum_{j=1}^J A_j^T y_j = 0, y_j \in \mathcal{K}_{m_j}, j = 1, 2, \dots, J \right\}. \end{aligned}$$

1.4. Contribution and organization

In this paper, we propose a penalty-type method for solving the inverse optimal value problem in second-order conic programming. Our contribution is to show that the above penalty term $\sum_{j=1}^J y_j^T (A_j x - b_j)$ is exact, which is different from the classical penalty-type method such as the quadratic penalty function method [16] and avoids the hurdles of penalty-type methods in the update strategy of the penalty parameter. On the other hand, our method also differs from the usual way to tackle the second-order conic complementarity system by using the SOC-cone functions [6,9], in which one needs more information on these functions such as continuous properties and differential formulae. Compared with the aforementioned methods, numerical results show that our method is more suitable to solve the above inverse optimal value problem (5).

The remainder of this paper is organized as follows. In Section 2, we present a penalty-type method for solving problem (5) and show the corresponding convergence results. We illustrate an example to verify these conclusions in Section 3. In Section 4, we conduct further numerical experiments to test the performance of our proposed method. Finally, some concluding remarks are drawn in Section 5.

2. Algorithm

In this section, we present a penalty-type method for solving the inverse optimal value problem in second-order conic programming and show the corresponding convergence results.

Throughout the rest of this paper, we make the following assumptions:

- (AS1) The feasible set $\mathcal{F}$ of problem (3) is nonempty bounded.
- (AS2) The set of cost vectors Θ is nonempty and compact.
- (AS3) An optimal value of problem (3) is attainable and its Slater's condition holds.

Based on these assumptions, a penalty-type method for solving problem (5) is presented in Algorithm 1. In order to show the convergence conclusion of Algorithm 1, we need to establish the following theoretical results.

Algorithm 1 A penalty-type method for solving problem (5).

Choose $\sigma_0 > 0, \tau > 1$ and set $k := 0$.
repeat
 Solve problem (6) to get the optimal solution $(\theta^{\sigma_k}, x^{\sigma_k}, y_1^{\sigma_k}, y_2^{\sigma_k}, \dots, y_J^{\sigma_k})$.
 Set $\sigma_{k+1} := \tau \sigma_k$ and $k := k + 1$.
until $\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) = 0$.
return $(\theta^{\sigma_k}, x^{\sigma_k})$.

Lemma 2.1. For a given positive scalar σ , suppose that the assumptions (AS1), (AS2) and (AS3) hold, then the following problem

$$\min_{x \in \mathcal{F}} P_\sigma(x) := \min_{(\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} \left\{ d(\theta^T x, \hat{v}) + \sigma \sum_{j=1}^J y_j^T (A_j x - b_j) \right\}$$

has optimal solutions. Hence, the penalized subproblem (6) has optimal solutions.

Proof. According to the assumptions (AS1), (AS2) and (AS3), we obtain

$$\begin{aligned} \min_{x \in \mathcal{F}} P_\sigma(x) &= \min_{x \in \mathcal{F}} \min_{(\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} \left\{ d(\theta^T x, \hat{v}) + \sigma \sum_{j=1}^J y_j^T (A_j x - b_j) \right\} \\ &\geq \min_{x \in \mathcal{F}} \left\{ \min_{(\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} d(\theta^T x, \hat{v}) + \min_{(\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} \sigma \sum_{j=1}^J y_j^T (A_j x - b_j) \right\} \\ &\geq \min_{x \in \mathcal{F}, (\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} d(\theta^T x, \hat{v}) + \min_{x \in \mathcal{F}, (\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} \sigma \sum_{j=1}^J y_j^T (A_j x - b_j). \end{aligned}$$

For fixed $\theta \in \Theta$, from the conic duality theorem and the KKT condition of problem (3), the problem (3) has optimal solutions and

$$\min_{x \in \mathcal{F}, (\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} \sigma \sum_{j=1}^J y_j^T (A_j x - b_j) = 0,$$

which imply that

$$\min_{x \in \mathcal{F}} P_\sigma(x) \geq \min_{x \in \mathcal{F}, (\theta, y_1, y_2, \dots, y_J) \in \mathcal{T}} d(\theta^T x, \hat{v}) = \min_{x \in \mathcal{F}, \theta \in \Theta} d(\theta^T x, \hat{v}) := f^*.$$

Then $P_\sigma(\cdot)$ is bounded from below by the finite optimal value f^* . Following the definitions of $P_\sigma(\cdot)$ and the sets $\mathcal{T}, \mathcal{F}$, the penalized subproblem (6) is equivalent to $\min_{x \in \mathcal{F}} P_\sigma(x)$. $\square$

Next, we will show that an exact solution of subproblem (6) can be obtained under a finite penalty parameter σ , where the associated penalty term $\sum_{j=1}^J y_j^T (A_j x - b_j)$ becomes zero.

Lemma 2.2. Suppose that assumptions (AS1), (AS2) and (AS3) hold. Let $\{(\theta^{\sigma_k}, x^{\sigma_k}, y_1^{\sigma_k}, y_2^{\sigma_k}, \dots, y_J^{\sigma_k})\}$ be a sequence of solutions of penalized subproblem (6) under the increasing positive penalty parameter sequence $\{\sigma_k\}$, then there exist a positive integer k_* and a positive scalar σ_* such that $\sigma_k \geq \sigma_*$ and $\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) = 0$ for all $k \geq k_*$. In this case, $(\theta^{\sigma_k}, x^{\sigma_k})$ solves the inverse optimal value problem (2).

Proof. Assume that $(\theta^*, x^*, y_1^*, y_2^*, \dots, y_J^*)$ is an optimal solution of problem (5), from the conic duality theorem and the KKT condition of problem (3), the associated complementarity slackness condition holds, i.e.,

$$(y_j^*)^T (A_j x^* - b_j) = 0, \quad j = 1, 2, \dots, J.$$

On the other hand, it follows from $(\theta^{\sigma_k}, x^{\sigma_k}, y_1^{\sigma_k}, y_2^{\sigma_k}, \dots, y_J^{\sigma_k})$ is optimal to the penalized subproblem (6) that

$$d((\theta^{\sigma_k})^T x^{\sigma_k}, \hat{v}) + \sigma_k \sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) \leq d((\theta^{\sigma_k})^T x^*, \hat{v}),$$

which yields

$$\begin{aligned} \sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) &\leq \frac{d((\theta^{\sigma_k})^T x^*, \hat{v}) - d((\theta^{\sigma_k})^T x^{\sigma_k}, \hat{v})}{\sigma_k} \\ &\leq \frac{d((\theta^{\sigma_k})^T x^{\sigma_k}, (\theta^*)^T x^*)}{\sigma_k} \end{aligned}$$

$$\leq \frac{M}{\sigma_k}, \quad (7)$$

where M is a positive constant, which is equal to

$$\max \{d((\theta^{\sigma_k})^T x^{\sigma_k}, (\theta^*)^T x^*) : \theta^{\sigma_k} \in \Theta, x^{\sigma_k} \in \mathcal{F}\}.$$

Assume that $\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j)$ is not equal to zero for any k . Due to the fact that $y_j^{\sigma_k} \in \mathcal{K}_{m_j}$ and $A_j x^{\sigma_k} - b_j \in \mathcal{K}_{m_j}$, then

$$\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) > 0, \quad \forall k.$$

Combining this observation with the relation (7), we deduce a contradiction. Hence, there exist a positive integer k_* and a positive scalar σ_* such that $\sigma_k \geq \sigma_*$ and $\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) = 0$ for all $k \geq k_*$. In addition, we have

$$\begin{aligned} & d((\theta^{\sigma_{k+1}})^T x^{\sigma_{k+1}}, \hat{v}) + \sigma_{k+1} \sum_{j=1}^J (y_j^{\sigma_{k+1}})^T (A_j x^{\sigma_{k+1}} - b_j) \\ & \geq d((\theta^{\sigma_{k+1}})^T x^{\sigma_{k+1}}, \hat{v}) + \sigma_k \sum_{j=1}^J (y_j^{\sigma_{k+1}})^T (A_j x^{\sigma_{k+1}} - b_j) \\ & \geq d((\theta^{\sigma_k})^T x^{\sigma_k}, \hat{v}) + \sigma_k \sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) \end{aligned} \quad (8)$$

where the first inequality follows from $\sigma_{k+1} = \tau \sigma_k > \sigma_k > 0$ and the second one uses the fact that $(\theta^{\sigma_k}, x^{\sigma_k}, y_1^{\sigma_k}, y_2^{\sigma_k}, \dots, y_J^{\sigma_k})$ solves the penalized subproblem (6). Similarly, we also obtain

$$\begin{aligned} & d((\theta^{\sigma_k})^T x^{\sigma_k}, \hat{v}) + \sigma_{k+1} \sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) \\ & \geq d((\theta^{\sigma_{k+1}})^T x^{\sigma_{k+1}}, \hat{v}) + \sigma_{k+1} \sum_{j=1}^J (y_j^{\sigma_{k+1}})^T (A_j x^{\sigma_{k+1}} - b_j) \end{aligned}$$

In the light of the above relation and (8), we have

$$(\sigma_{k+1} - \sigma_k) \left[\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) - \sum_{j=1}^J (y_j^{\sigma_{k+1}})^T (A_j x^{\sigma_{k+1}} - b_j) \right] \geq 0,$$

which deduces that

$$\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) \geq \sum_{j=1}^J (y_j^{\sigma_{k+1}})^T (A_j x^{\sigma_{k+1}} - b_j).$$

From the relation (8), we obtain

$$\begin{aligned} & d((\theta^{\sigma_{k+1}})^T x^{\sigma_{k+1}}, \hat{v}) - d((\theta^{\sigma_k})^T x^{\sigma_k}, \hat{v}) \\ & \geq \sigma_k \left[\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) - \sum_{j=1}^J (y_j^{\sigma_{k+1}})^T (A_j x^{\sigma_{k+1}} - b_j) \right] \\ & \geq 0. \end{aligned}$$

From the above properties of penalized problem (6), we can find that the upper level objective $d(\theta^T x, \hat{v})$ and the objective function of penalized subproblem (6) are both monotonically nondecreasing in the penalty parameter σ_k . Then we can get the optimal solutions of problem (2) by solving problem (6). Furthermore, if the relation $\sum_{j=1}^J (y_j^{\sigma_k})^T (A_j x^{\sigma_k} - b_j) = 0$ occurs, then $(\theta^{\sigma_k}, x^{\sigma_k}, y_1^{\sigma_k}, y_2^{\sigma_k}, \dots, y_J^{\sigma_k})$ solves problem (5), which is equivalent to the original inverse optimal value problem (2) under the assumption (AS3). $\square$

Remark 2.1. From Lemma 2.2, the bilinear term $\sum_{j=1}^J y_j^T (A_j x - b_j)$ is an exact penalty function of problem (5).

After these preparations, we conclude this section in the following theorem about the convergence conclusion of Algorithm 1.

Theorem 2.1. Suppose that assumptions (AS1), (AS2) and (AS3) hold. Then the sequence $\{(\theta^{\sigma_k}, x^{\sigma_k})\}$, generated by Algorithm 1, converges to the optimal solution of problem (2).

Proof. If the iterations of $\{(\theta^{\sigma_k}, x^{\sigma_k})\}$ are finite, from the stop criterion of Algorithm 1, the last point in the sequence $\{(\theta^{\sigma_k}, x^{\sigma_k})\}$ solves problem (2).

On the other hand, if iterations of $\{(\theta^{\sigma_k}, x^{\sigma_k})\}$ are infinite, we can deduce that $\{(\theta^{\sigma_k}, x^{\sigma_k})\} \subset \Theta \times \mathcal{F}$. Following from assumptions (AS1) and (AS2), there exists an accumulation point (θ^*, x^*) of the sequence $\{(\theta^{\sigma_k}, x^{\sigma_k})\}$ that lies in the set $\Theta \times \mathcal{F}$. For the point (θ^*, x^*) , it follows from assumption (AS3) that there exist $y_1^*, y_2^*, \dots, y_J^*$ such that $(\theta^*, x^*, y_1^*, y_2^*, \dots, y_J^*)$ solves problem (6). From Lemma 2.2, $\sum_{j=1}^J (y_j^*)^T (A_j x^* - b_j) = 0$ and (θ^*, x^*) solves the inverse optimal value problem (2). $\square$

3. Example

In this section, we present an example to illustrate these results established in Section 2.

Example 3.1. Consider the following second-order conic programming problem

$$\begin{aligned} \min \quad & \theta_1 x_1 + \theta_2 x_2 \\ \text{s.t.} \quad & \begin{pmatrix} 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} - \begin{pmatrix} -2 \\ 0 \\ 0 \end{pmatrix} \in \mathcal{K}_3. \end{aligned}$$

We set the parameter set Θ and the target value $\hat{v}$ as

$$\Theta = \{(\theta_1, \theta_2) : -1 \leq \theta_1 \leq 1, 0.5 \leq \theta_2 \leq 1.5\}, \quad \hat{v} = -2.$$

In our test, we choose two types of metric distance function $d(\cdot, \cdot)$ in the penalized subproblem (6), i.e. $d_1(u, v) = (u - v)^2$ and $d_2(u, v) = |u - v|$. Then, the corresponding penalized inverse optimal value problems are given by

$$\begin{aligned} \min \quad & (\theta_1 x_1 + \theta_2 x_2 + 2)^2 + \sigma(2y_1 + x_1 y_2 + x_2 y_3) \\ \text{s.t.} \quad & x_1^2 + x_2^2 \leq 4, \quad y_2^2 + y_3^2 \leq y_1^2, \quad y_1 \geq 0, \\ & -1 \leq \theta_1 \leq 1, \quad 0.5 \leq \theta_2 \leq 1.5 \end{aligned} \quad (9)$$

and

$$\begin{aligned} \min \quad & t + \sigma(2y_1 + x_1 y_2 + x_2 y_3) \\ \text{s.t.} \quad & x_1^2 + x_2^2 \leq 4, \quad y_2^2 + y_3^2 \leq y_1^2, \quad y_1 \geq 0, \\ & -1 \leq \theta_1 \leq 1, \quad 0.5 \leq \theta_2 \leq 1.5, \\ & -t \leq \theta_1 x_1 + \theta_2 x_2 + 2 \leq t, \quad t \geq 0. \end{aligned} \quad (10)$$

During the process of Algorithm 1, we set $\sigma_0 = 100$, $\tau = 1.1$ and use JuMP and Ipopt packages [17] in Julia as the solver of problems (9) and (10). The associated optimal solution is $\theta^{\sigma_0} = (0, 1)^T$, $x^{\sigma_0} = (0, -2)^T$ and $y^{\sigma_0} = (0, 0, 0)^T$. In this case, we obtain $(y^{\sigma_0})^T (A x^{\sigma_0} - b) = 0$. It follows from Algorithm 1 and Lemma 2.2 that the optimal solution of problems (9) and (10) is $\theta^* = (0, 1)^T$ and $x^* = (0, -2)^T$.

Remark 3.1. In Example 3.1, Algorithm 1 only needs one iteration. The main reason why such thing happens is because our penalty term is exact and an appropriate penalty parameter is chosen.

4. Implementation details and numerical results

In this section, we conduct further numerical experiments to test the performance of our proposed algorithm in Section 2 by comparing it with the quadratic penalty method [16] and the SOC-cone function based method [6,9] for inverse optimal value problems in SOCP under different values of sizes m, n . All experiments are run on a 64-bit PC with an Intel (R) Core(TM) i9-12900 of 2.40 GHz CPU and 32.00 GB of RAM equipped with Windows 11 operating system.

4.1. Implementation details

- *Subproblems for different algorithms:* In our test, the subproblem of the quadratic penalty method is given by

$$\begin{aligned} \min_{\theta, x, y_1, y_2, \dots, y_J} \quad & d(\theta^T x, \hat{v}) + \frac{\sigma}{2} \sum_{j=1}^J (y_j^T (A_j x - b_j))^2 \\ \text{s.t.} \quad & \theta - \sum_{j=1}^J A_j^T y_j = 0, \quad \theta \in \Theta, \\ & A_j x - b_j \in \mathcal{K}_{m_j}, \quad y_j \in \mathcal{K}_{n_j}, \quad j = 1, 2, \dots, J. \end{aligned}$$

Table 1
Numerical results for the objective value $d_1(\theta^T x, \hat{v})$ and its residual.

n	m	Alg1		Alg2		Alg3	
		obj ₁	Res	obj ₁	Res	obj ₁	Res
5	100	1.181e-11	2.239e-07	4.488e-11	7.873e-06	5.205e-11	9.739e-07
	200	1.337e-11	2.379e-07	4.792e-11	8.417e-06	4.192e-11	3.596e-07
	300	1.600e-11	3.223e-07	5.635e-11	6.860e-06	1.653e-11	6.878e-07
	400	2.084e-11	4.753e-07	8.620e-11	1.097e-06	8.529e-11	4.608e-07
	500	2.888e-11	5.687e-07	8.973e-10	3.571e-05	8.715e-10	7.617e-07
10	100	4.367e-11	3.084e-07	8.494e-11	7.032e-06	7.174e-11	9.919e-07
	200	1.973e-11	2.699e-07	6.977e-11	8.916e-06	4.975e-11	7.673e-07
	300	5.090e-11	3.752e-07	7.714e-11	9.792e-06	5.500e-11	8.699e-07
	400	5.118e-11	4.760e-07	8.708e-11	6.809e-05	9.028e-11	1.031e-06
	500	7.575e-11	6.840e-07	9.094e-10	7.899e-05	8.927e-10	1.753e-06

Table 2
Numerical results for CPU-time under the objective value $d_1(\theta^T x, \hat{v})$.

n	m	Alg1-time	Alg2-time	Alg3-time
5	100	0.142 s	0.116 s	1.049 s
	200	0.279 s	0.372 s	5.599 s
	300	0.624 s	1.191 s	19.356 s
	400	0.753 s	1.974 s	55.701 s
	500	1.966 s	2.771 s	98.630 s
10	100	0.287 s	0.211 s	1.069 s
	200	0.571 s	0.748 s	6.364 s
	300	1.759 s	1.927 s	39.990 s
	400	1.826 s	2.380 s	57.602 s
	500	3.581 s	5.115 s	109.091 s

The subproblem of the SOC-cone function based method has the following form

$$\begin{aligned} \min_{\theta, x, y_1, y_2, \dots, y_J} \quad & d(\theta^T x, \hat{v}) + \frac{\sigma}{2} \sum_{j=1}^J (\phi_\mu(A_j x - b_j, y_j))^2 \\ \text{s.t.} \quad & \theta - \sum_{j=1}^J A_j^T y_j = 0, \quad \theta \in \Theta, \end{aligned}$$

where $\phi_\mu(u, v)$ is the smoothing Fischer–Burmeister function [9] with $\mu := 0.001$. At the same time, the subproblem of Algorithm 1 is defined as in (6). All these problems are implemented in Julia (Version 1.9) by using the package ECOS [8] to generate the optimal value of problem (1) and the package Ipopt [17] as the main solver.

- **Data generation:** We first set random $m \times n$ matrices A_j and $m \times 1$ vectors b_j in problem (1). Next, we use the package ECOS to obtain the corresponding optimal value of problem (1) with a given random $n \times 1$ vector $\hat{v}$ and set it to be the value of $\hat{v}$ in the subproblem (6). The parameter set Θ is given by $\Theta := \{\theta : \hat{\theta} - 0.1 \cdot l \leq \theta \leq \hat{\theta} + 0.1 \cdot u\}$, where l and u are random $n \times 1$ vectors with entries in $[0, 1]$.
- **Other settings:** Set the parameters $\sigma_0 = 100$, $\tau = 1.1$ and use the residual of stopping criterion $\text{Res} := |(y^{\sigma_k})^T (Ax^{\sigma_k} - b)| \leq 10^{-4}$. Similar as Example 3.1, we choose two types of metric distance function $d(\cdot, \cdot)$ in the above subproblems, i.e. $d_1(u, v) := (u - v)^2$ and $d_2(u, v) := |u - v|$. In the following tables, we denote obj_1 and obj_2 the value of $d_1(\theta^T x, \hat{v})$ and $d_2(\theta^T x, \hat{v})$ as the objective functions in problem (5).

4.2. Comparison with different algorithms

Now, we compare our algorithm with the quadratic penalty method and the SOC-cone function based method, which are respectively denoted by Alg1, Alg2 and Alg3 in the following tables.

Table 1 and Table 2 show that the solvability of our method and others for the instances with $m = 100, 200, 300, 400, 500$ under the choice of metric distance function d_1 in the aforementioned subproblems. Numerical results show that our method is more efficient in terms of CPU-time and get solutions with better objective value and lower residuals.

Numerical results in Table 3 and Table 4 indicate that our method also costs less computing time for the instances with different sizes under another choice of metric distance function d_2 in the aforementioned subproblems.

Table 3
Numerical results for the objective value $d_2(\theta^T x, \hat{v})$ and its residual.

n	m	Alg1		Alg2		Alg3	
		obj ₂	Res	obj ₂	Res	obj ₂	Res
5	100	3.233e-11	9.999e-08	3.993e-9	1.444e-06	2.684e-10	8.970e-07
	200	1.647e-10	3.294e-07	4.689e-9	4.139e-06	3.166e-10	9.996e-07
	300	2.923e-10	3.908e-07	5.389e-9	5.464e-06	4.576e-10	1.899e-06
	400	3.780e-10	4.308e-07	5.601e-9	5.882e-06	4.872e-10	5.674e-06
	500	4.132e-10	7.019e-07	7.290e-9	5.547e-05	5.658e-10	8.960e-06
10	100	4.335e-11	1.310e-07	4.835e-9	4.228e-06	1.557e-10	9.815e-07
	200	6.557e-10	4.773e-07	5.106e-9	4.384e-06	4.050e-10	1.703e-06
	300	6.770e-10	5.094e-07	5.180e-9	6.983e-06	5.749e-10	1.917e-06
	400	7.008e-10	6.333e-07	6.872e-9	7.932e-06	1.194e-09	6.161e-06
	500	7.816e-10	7.311e-07	7.444e-9	8.651e-05	7.024e-09	7.158e-05

Table 4
Numerical results for CPU-time under the objective value $d_2(\theta^T x, \hat{v})$.

n	m	Alg1-time	Alg2-time	Alg3-time
5	100	0.131 s	0.154 s	2.300 s
	200	0.279 s	0.368 s	9.901 s
	300	0.760 s	1.255 s	27.423 s
	400	0.790 s	1.721 s	45.141 s
	500	2.021 s	3.026 s	104.993 s
10	100	0.187 s	0.213 s	0.981 s
	200	0.713 s	0.871 s	9.923 s
	300	1.599 s	1.738 s	30.195 s
	400	1.832 s	4.154 s	56.619 s
	500	3.665 s	4.884 s	128.197 s

Table 5
Numerical results for high-dimensional problems.

n	m	Alg1-time1	Alg1-time2
5	1000	3.214 s	3.567 s
	2000	10.969 s	11.754 s
	3000	23.391 s	24.278 s
10	1000	6.253 s	7.029 s
	2000	22.591 s	27.953 s
	3000	46.039 s	48.907 s

4.3. Comparison with different choices of metric distance function for high-dimensional problems

In this subsection, we report numerical experiments of our method under different choices of metric distance function similar as Example 3.1, in which the CPU-time under the metric distance functions d_1 and d_2 are respectively denoted by Alg1-time1 and Alg1-time2.

Numerical results in Table 5 show that our algorithm can solve the high-dimensional inverse optimal value problems in SOCP under both choices of metric distance functions d_1 and d_2 , in which the first one has a better performance on CPU-time.

5. Concluding remarks

In this paper, we study a penalty-type method for minimizing a class of inverse optimal value problems in second-order conic programming. Under mild conditions, we show that any cluster point of the sequence generated by the proposed approach is an optimal solution of the original inverse optimal value problem. In particular, we also show that the associated penalty term is exact. Numerical results show that our algorithm is suitable to solve the given inverse optimal value problem under different metric distances between the objective function of the forward problem and a target value. We believe that the framework of our algorithm can be adapted to solve other types of inverse optimal value problems with general conic constraints. We leave these further discussions as our future work.

Acknowledgements

The first author's work is supported by National Natural Science Foundation of China (Grant Number: 11601389). The fourth author's work is supported by Postgraduate Research Innovation Project of Tianjin Normal University (Grant Number: 2023KYCX054Y). The fifth author's work is supported by Natural Science Foundation of Tianjin (Grant Number: 19JCQNJC03700).

References

- [1] S. Ahmed, Y. Guan, The inverse optimal value problem, *Math. Program.* 102 (2005) 91–110.
- [2] R. Ahuja, J. Orlin, Inverse optimization, *Oper. Res.* 49 (2001) 771–783.
- [3] R. Ahuja, J. Orlin, Combinatorial algorithms for inverse network flow problems, *Networks* 40 (2002) 181–187.
- [4] W. Burton, P. Toint, On an instance of the inverse shortest paths problem, *Math. Program.* 53 (1992) 45–61.
- [5] M. Cai, X. Yang, J. Zhang, The complexity analysis of the inverse center location problem, *J. Glob. Optim.* 15 (1999) 213–218.
- [6] J.S. Chen, P. Tseng, An unconstrained smooth minimization reformulation of second-order cone complementarity problem, *Math. Program.* 104 (2005) 293–327.
- [7] Z. Drezner, H.W. Hamacher, *Facility Location Applications and Theory*, Springer, New York, 2004.
- [8] A. Domahidi, E. Chu, S. Boyd, ECOS: an SOCP solver for embedded systems, <https://doi.org/10.23919/ECC.2013.6669541>, 2013.
- [9] M. Fukushima, Z.Q. Luo, P. Tseng, Smoothing functions for second-order-cone complementarity problems, *SIAM J. Optim.* 12 (2002) 436–460.
- [10] C. Heuberger, Inverse combinatorial optimization: a survey on problems, methods and results, *J. Comb. Optim.* 8 (2004) 329–361.
- [11] G. Iyengar, W. Kang, Inverse conic programming and applications, *Oper. Res. Lett.* 33 (2005) 319–330.
- [12] J. Jia, X. Guan, H. Wang, B. Zhang, P.M. Pardalos, Combinatorial algorithms for solving the restricted bounded inverse optimal value problem on minimum spanning tree under weighted l_∞ norm, *J. Comput. Appl. Math.* 419 (2023) 114754.
- [13] M. Lobo, L. Vandenberghe, S. Boyd, H. Lebert, Applications of second-order cone programming, *Linear Algebra Appl.* 284 (1998) 193–228.
- [14] Y. Lu, Y. Ge, L. Zhang, An alternating direction method for solving a class of inverse semidefinite quadratic programming problems, *J. Ind. Manag. Optim.* 12 (2016) 317–336.
- [15] Y. Lu, M. Huang, Y. Zhang, J. Gu, A nonconvex ADMM for a class of sparse inverse semidefinite quadratic programming problems, *Optimization* 68 (2019) 1075–1105.
- [16] J. Nocedal, S.J. Wright, *Numerical Optimization*, Springer, New York, 1999.
- [17] A. Wächter, T. Biegler, On the implementation of a primal-dual interior point filter line search algorithm for large-scale nonlinear programming, *Math. Program.* 106 (2006) 25–57.
- [18] H. Wang, X. Guan, Q. Zhang, B. Zhang, Capacitated inverse optimal value problem on minimum spanning tree under bottleneck Hamming distance, *J. Comb. Optim.* 41 (2021) 861–887.
- [19] J. Wu, Y. Zhang, L. Zhang, Y. Lu, A sequential convex program approach to an inverse linear semidefinite programming problem, *Asia-Pac. J. Oper. Res.* 33 (2016) 1–26.
- [20] X. Xiao, L. Zhang, J. Zhang, A smoothing Newton method for a type of inverse semidefinite quadratic programming problem, *J. Comput. Appl. Math.* 223 (2009) 485–498.
- [21] X. Xiao, L. Zhang, J. Zhang, On convergence of augmented Lagrange method for inverse semidefinite quadratic programming problems, *J. Ind. Manag. Optim.* 5 (2009) 319–339.
- [22] B. Zhang, X. Guan, Q. Zhao, Inverse optimal value problem on minimum spanning tree under unit l_∞ norm, *Optim. Lett.* 14 (2020) 2301–2322.
- [23] B. Zhang, X. Guan, P.M. Pardalos, H. Wang, Q. Zhao, Y. Liu, S. Chen, The lower bounded inverse optimal value problem on minimum spanning tree under unit l_∞ norm, *J. Glob. Optim.* 79 (2021) 757–777.
- [24] J. Zhang, Z. Liu, A further study on inverse linear programming problems, *J. Comput. Appl. Math.* 106 (1999) 345–359.
- [25] J. Zhang, Z. Liu, Calculating some inverse linear programming problems, *J. Comput. Appl. Math.* 72 (1996) 261–273.
- [26] J. Zhang, Z. Liu, Z. Ma, Some reverse location problems, *Eur. J. Oper. Res.* 124 (2000) 77–88.
- [27] J. Zhang, Z. Ma, Solution structure of some inverse combinatorial optimization problems, *J. Comb. Optim.* 3 (1999) 127–139.
- [28] J. Zhang, L. Zhang, An augmented Lagrangian method for a class of inverse quadratic programming problems, *Appl. Math. Optim.* 61 (2010) 57–83.
- [29] Q. Zhao, X. Guan, J. Jia, X. Qian, P.M. Pardalos, The restricted inverse optimal value problem on shortest path under l_1 norm on trees, *J. Glob. Optim.* 86 (2023) 251–284.